Elasticsearch 作为 ECDM 数据库的利弊分析

1. 按Raynor的说法（3.23）
2. 用 Elasticsearch 当数据库的好处
   2.1. 强大的全文搜索能力
   2.2. Schema-free / 灵活的文档模型：
   2.3. 水平可扩展的分布式架构：
      2.3.1. 主要目的
      2.3.2. 分片到底是怎么切的
      2.3.3. “加节点 != 一定更快”
      2.3.4. 和PostgreSQL 相比
   2.4. 内建乐观并发控制（OCC）
      2.4.1. 概念
      2.4.2. 字段实现
      2.4.3. update() 为什么和 OCC 有关系
      2.4.4. 自动重试机制
   2.5. 近实时搜索
3. Elasticsearch 相比关系型数据库的弊端
   3.1. 无 ACID 事务
   3.2. 无 JOIN查询
   3.3. Near-real-time (NRT) 延迟
   3.4. 无严格 Schema 约束
   3.5. Schema 变更困难
   3.6. 10K 结果窗口限制
      3.6.1. 传统分页
      3.6.2. 新模式
   3.7. 往已有 mapping 里追加新字段，不能"改字段"
      3.7.1. 三种替代方案
         3.7.1.1. 重建索引（最彻底、最推荐）
         3.7.1.2. 使用 _source 过滤（逻辑删除）

3.7.1.3. 直接置之不理
4. 针对弊端，ECDM 采用的缓解方案
4.1. 缺乏 ACID 事务 → 乐观并发控制（OCC）重试 + try-create/catch-update 模式
4.1.1. 乐观并发控制（OCC）+ 自动重试
4.1.2. createOrUpdate 的 try/catch 补偿模式
4.2. 反范式化（Denormalization）— 解决"无 JOIN"
4.2.1. 举例一：在schema升级场景下，自定义 FOREIGN handler
4.2.2. 举例二：事件监听机制
4.3. 可配置的 Refresh 策略 — 解决"NRT 延迟"
4.4. Postgres Key-Value Store — 补充 ES 在事务性操作上的不足
4.5. 事件监听机制 — 弥补无触发器/外键
5. 仍未解决的弊端
6. Lucene
6.1. searcher / segment 是什么
6.1.1. segment
6.1.2. searcher
6.1.3. 整体流程

1. 按Raynor的说法（3.23）

是从Day1开始的设计

- 因为PPDM的前身，主要存储copy和asset这种需要多次读、但很少写的数据，就这一点而言，ES的query读是有优势的

但随着PPDM的发展

- 出现了像activity这样会多次写的数据，这样ES的弊端就显现（速度慢 + 只能实现最终一致性）。计划利用k-v store来替代，但在产品中优先级不够
- 什么是优先级更高的？切了activity v3之后，http的IO体量太大，要优先解决这个问题

改进方法

- 利用缓存Redis，把需要经常改写的数据放在本地，不需要反复的走完整的IO链路

2. 用 Elasticsearch 当数据库的好处

2.1. 强大的全文搜索能力

ES 天生擅长全文检索和复杂查询，ECDM 作为数据保护管理平台，需要对大量资产(asset)、策略(policy)、任务(job)等进行灵活的多字段组合过滤和搜索。代码中 `Query`/`Filter`/`Filters` 体系非常完善

- 所有 85 个索引的 mapping 大量使用 keyword + case_insensitive normalizer，支持高效的精确匹配和排序
- nested 类型字段（如 connectionInfos, discoveryJobs）支持嵌套对象查询

2.2. Schema-free / 灵活的文档模型：

JSON 文档存储天然适合存储异构的实体（不同类型的存储设备、不同类型的数据源等），不需要严格的表结构。

- ES 的动态 mapping 允许增量添加字段而不需要停机迁移

2.3. 水平可扩展的分布式架构：

- ES天生就是分布式系统（ES 集群可以水平扩展），当数据量、请求量变大时，可以通过**增加节点**来扩容，而不是只能把单机做得越来越大。

2.3.1. 主要目的

1. 提升处理能力：
   - 同时处理更多请求，读写更多数据，查询速度更稳定（并发读能力）
   - 数据会分散到不同节点上，不同节点可以并行处理请求
   - 写入能力也能提升，但是没有读这么明显。因为写入还要考虑副本同步、刷新、协调节点开销
2. 提升容量：
   - 随着 ECDM 数据越来越多，单机磁盘、内存、CPU 都会不够
   - 增加节点能让数据库能继续装更多数据，而不是很快就到上限
3. 提高可用性（不容易挂）：
   - ES 是分布式的，通常会有：主分片，副本分片
   - 如果一个节点坏了，其他节点上的副本还能继续提供服务，系统不一定马上整体不可用
     - 支持“并行查询”

> 比如你查：
> “找所有 `state=RUNNING` 的 job”
> ES 会：
> - 去分片 0 查
> - 去分片 1 查
> - 去分片 2 查
> - 最后把结果合并返回

2.3.2. 分片到底是怎么切的

- 一个索引里的很多文档，会按照规则（通常是 `_id`，算一个哈希值）分散到不同分片里

  > 比如可能变成：
  > - **分片 0**：job1, job4
  > - **分片 1**：job2, job5
  > - **分片 2**：job3
  >
  > 然后再把这些分片放到不同节点上：
  > - **节点 A**：分片 0
  > - **节点 B**：分片 1
  > - **节点 C**：分片 2

- 副本是什么
  - 复制一份 主分片的信息，在另外一个（数量是可配的）节点上

    > 主分片 0 在节点 A，副本 0 在节点 B
    > 主分片 1 在节点 B，副本 1 在节点 C
    > 主分片 2 在节点 C，副本 2 在节点 A

  - 某个节点坏了，还有副本顶上 + 读请求可以分摊到副本上

2.3.3. “加节点 != 一定更快”

- 额外开销：分片协调、节点间通信、数据复制、结果汇总
- 数据倾斜：有的节点很忙、有的节点很闲
- 管理成本（更复杂）：分片管理、故障恢复、内存压力和 JVM 调优

2.3.4. 和PostgreSQL 相比

- PostgreSQL 的经典形态是 **单机关系型数据库优先**，分布式不是它最原生、最自然的能力。
  - **一个主库，**可以有多个只读从库（读扩容容易），但**写入一般还是集中在主库**（写扩容很难）
- PostgreSQL 有**强事务**，分布式拆分就越难

2.4. 内建乐观并发控制（OCC）

- **代码证据**：EsBaseDao 的 update() 方法利用 ES 的 `_seq_no` 和 `_primary_term` 实现 OCC
- **自动重试机制**：OCC_VERSION_CONFLICT_MAX_RETRIES = 7

2.4.1. 概念

当两个人几乎同时去修改同一条 ES 数据时，系统要防止“后写的人把前写的人覆盖掉”，这就叫乐观并发控制（OCC, Optimistic Concurrency Control）

  - 乐观？系统默认认为冲突不会经常发生，所以平时不加锁，提交时检查版本，等冲突了再报错/重试。
  - 悲观锁：一开始就先锁住，不让别人改
- OCC 的思路是：你修改数据时，必须证明你改的是“你当初读到的那个版本”。 如果版本已经变了，说明别人先改过了，那你的更新就不能直接写进去。
- 主要解决：并发更新冲突、丢失更新

2.4.2. 字段实现

- `_seq_no` ：这条数据的“修改序号”
- `_primary_term` ：主分片任期，保证主分片切换时版本判断仍然正确

在 EsBaseDao 里，代码会把 ES 的这两个值拼成一个 `occVersion`

- `entity.getOccVersion()`

2.4.3. update() 为什么和 OCC 有关系

1. 先读出数据库里的当前数据：拿到当前实体和它的 `occVersion`
2. 检查你传进来的版本是不是旧的
   - 如果你传入的 `entity.getOccVersion()` 和数据库当前版本不一致，就说明：
     - 你改的是旧数据
     - 有人已经先一步改过了
   - 这时就抛出： `OccVersionConflictException`

2.4.4. 自动重试机制

- 如果更新时发现版本冲突，系统不会立刻放弃，而是可以重新读取最新数据，再试一次，最多试 7

次。

- 解决方法：
  a. 发现版本冲突，再读一次最新版本
  b. 基于最新版本重新计算 – 重新应用你的修改意图 – 只改部分

  ```json
  {
    "progress": 60,
    "owner": "Tom"
  }
  ```

  这样会把别人的 `Jerry` 覆盖回去。

  正确重试方式

  重新拿最新对象，再应用修改逻辑：

  - 最新对象是 `owner=Jerry`
  - 你的逻辑只是改 `progress`

  所以最终能同时保留：

  - 你的修改
  - 别人的修改

  c. 再尝试更新

2.5. 近实时搜索

- 最终一致性：写入后短时间内（1s）即可搜索到，对很多场景已经足够
- 与 实时一致性 形成对比：关系型数据库的优势，不存在"写了但搜不到"的窗口期

3. Elasticsearch 相比关系型数据库的弊端

3.1. 无 ACID 事务

- 没有多文档事务，无法回滚 – 就是“不提供”，只能独立操作
- 跨索引操作无法保证事务性：例如创建保护策略同时更新关联资产，若中途失败会导致数据不一致

3.2. 无 JOIN查询

- 无法做关系型的表连接

- 关系查询能力弱——模拟外键

3.3. Near-real-time (NRT) 延迟

- 写入后不立刻可搜索，默认1s刷新间隔
- EsBaseDao.read() 使用 GET /{index}/_doc/{id} 可以实时读取，但 search() 依赖刷新
  - read()是by-id get：直接从主分片/副本分片的当前数据里拿到
    - “实时”：比 search 更接近实时，不是传统数据库那种“事务提交后全局强一致”

```java
GET /index_data_source/_doc/123
```

  - search()依赖的是“refresh 后的 segment / searcher”（refer to 5.1），只有在 refresh 之后，才会被纳入新的可搜索视图

```java
POST /index_data_source/_search
{
  "query": {
    "term": {
      "name.keyword": "vm-01"
    }
  }
}
```

3.4. 无严格 Schema 约束

- 无法保证数据完整性和字段约束
  - 没有NOT NULL约束 – 只能在JAVA应用层人工加逻辑校验
  - 没有UNIQUE约束 – ES 只有 `_id` 是唯一的。其他字段**没有唯一性约束**
  - 没有外键约束 – 所有的引用完整性都由应用层维护
- 做了什么弥补?

5. ECDM 做了什么弥补？

ECDM 确实在 `v_19_5_0` 开始设置了 `"dynamic": "strict"`：

json@c:\ecdm-0910\cs\data-access\data-access-elasticsearch-init\src\main\resources\es_schema\v_19_5_0\patch_063\d

```
{
   "dynamic":"strict",
   ...
}
```

这意味着：

- **不允许写入 mapping 中未定义的字段**
- 如果你尝试写入一个 mapping 里没有的字段，ES 会拒绝

但 `dynamic: strict` **只能防止多写字段，不能防止少写字段**。也就是说：

场景	`dynamic: strict` 能防住吗？
写入未定义的字段	✅ 能
缺少必填字段	❌ 不能
字段值不合法（如 state="INVALID"）	❌ 不能
引用了不存在的外键	❌ 不能
重复的业务唯一键	❌ 不能

3.5. Schema 变更困难

- 需要提供专门的 `elasticsearch-schema-upgrade` 模块，自定义迁移框架
- **对比**：关系型数据库使用 Flyway/Liquibase 的 SQL DDL 迁移，一条 `ALTER TABLE ADD COLUMN` 即可完成的事

3.6. 10K 结果窗口限制

- ES 的默认分页限制（`max_result_window = 10000`）对大规模资产管理场景造成了**实际的工程困难**
- 需要ECDM配置workaround：
  - 不再依赖 ES 那种受 10K 限制的深分页方式
  - **新方式**：二者叠加
    i. 第一层：服务端返回一个 `queryState`，客户端拿着这个 `queryState` 继续往后翻
    ii. 第二层：如果你要取超过 10K 的数据，ECDM 不允许直接把所有结果都塞进内存里的一个 `List`，边取边处理 / 边写文件

3.6.1. 传统分页

传统分页一般是这种思路：

- 第 1 页：取第 1 ~ 100 条
- 第 2 页：取第 101 ~ 200 条
- …
- 第 101 页：取第 10001 ~ 10100 条

这在 ES 里通常对应：

- `from`
- `size`

问题是 ES 默认不让你随便翻到特别深的位置。
默认 `max_result_window = 10000`，所以超过 10K 就容易报错。

3.6.2. 新模式

- 也就是：**不是告诉服务端“我要第 101 页”**，而是告诉服务端：**“请从你上次给我的位置继续往后取”**
- 避开了：**ES 默认深分页的实现方式**

第一次请求

- 客户端传 `queryState = BEGIN`

服务端返回

- 返回第一页数据
- 同时返回一个新的 `queryState`

第二次请求

- 客户端把这个新的 `queryState` 再传回去

一直循环

- 直到服务端返回 `queryState = END`

- 弊端：只能依次向后翻，不能跳到指定页

3.7. 往已有 mapping 里追加新字段，不能"改字段"

- Elasticsearch (ES) **不支持**直接删除已存在的字段（Field）定义
  - Elasticsearch 的底层是 Lucene。为了实现极高的搜索性能，Lucene 会将数据写入**不可变的倒排索引**（Inverted Index）文件中。

3.7.1. 三种替代方案

3.7.1.1. 重建索引（最彻底、最推荐）

1. 创建一个新索引，定义新的 mapping
2. 把旧索引的所有数据 reindex 到新索引

```Java
POST _reindex
{
  "source": { "index": "old_index" },
  "dest": { "index": "new_index" }
}
```

3. 删除旧索引
4. 用 alias 把新索引指向旧名字 ->应用层不用改代码
   - Alias 是 ES 里的索引别名。代码只认这个外号，背后实际指向哪个索引可以随时切换。

```Java
POST /_aliases
{
  "actions": [
    { "remove": { "index": "index_data_source",    "alias": "index_data_source" }},
    { "add":    { "index": "index_data_source_v2",  "alias": "index_data_source" }}
  ]
}
```

   - 操作风险
     - reindex 期间，新写入旧索引的数据可能丢失（除非做双写）
     - reindex 时间长，数据量大的索引可能需要几分钟到几小时
     - 需要额外磁盘空间，新旧索引同时存在

3.7.1.2. 使用 _source 过滤（逻辑删除）

- 如果你只是不想在查询结果中看到这个字段，但不在乎它是否占用磁盘空间，可以在 Mapping 中禁用它，或者在查询时过滤掉。
  - 更新 Mapping (只能新增，不能改旧)： 你可以将字段的 **enabled 设为 false**（仅限 object 类型），或者在查询时使用 _source 排除。
  - 注意： 这种方法并不会减小索引体积，只是让该字段“不可查”。

3.7.1.3. 直接置之不理

- 只要新写入的文档不再包含这个字段，它就不会再占用后续新增段的空间。这种做法虽然不优雅，但对于 PB 级的大索引来说，比 _reindex 的成本低得多。

4. 针对弊端，ECDM 采用的缓解方案

4.1. 缺乏 ACID 事务 → 乐观并发控制（OCC）重试 + try-create/catch-update 模式

4.1.1. 乐观并发控制（OCC）+ 自动重试

```java
// 最底层：每次实际 update ES 前先 read 最新版本，利用 _seq_no + _primary_term 做乐观锁
final UpdateResponse<Void> response = esClient.update(builderObj -> builderObj
    .index(index)
    .id(documentId)
    .doc(changedDocument.getSourceAsObjectNode())
    .ifPrimaryTerm(document.getPrimaryTerm())
    .ifSeqNo(document.getSequenceNumber()),
    Void.class
);
// 如果版本冲突（HTTP 409），自动重试，最多 7 次
```

4.1.2. createOrUpdate 的 try/catch 补偿模式

```java
public boolean createOrUpdate(SecurityContext securityContext, T entity) {
    // ...
    create(securityContext, entity);
    return true;
// 自定义Exception: 如果失败原因是“重复键”
    // 如果发现对象（同一个 id 已存在）已经存在，那就不创建了，转而更新它。
} catch (DuplicateKeyException e) {
    update(securityContext, entity);
    return false;
}

// 此时 update 有两层
// 第一层：调用方版本 vs 数据库当前版本
// 调用方版本：传入update的版本
// 数据库当前版本：刚从 ES 读出来的最新版本
// 第二层：ES 原生 OCC 提交保护 （_seq_no + _primary_term）
```

4.2. 反范式化（Denormalization）— 解决"无 JOIN"

- “逻辑关联”–> 在应用层自己模拟“关联查询”，实际会拆成多次 ES 查询
  - 对比数据库，有“外键约束”

4.2.1. 举例一：在schema升级场景下，自定义 FOREIGN handler

你提到的第二部分，是 **schema 升级场景** 下的同一个问题。
在 schema upgrade 配置里，ECDM 定义了多种字段迁移方式：

- `PLAIN`
- `JSON`
- `FOREIGN`
- `STAGE`

其中 `FOREIGN` 的含义就是：

> *“当前索引里的这个字段，不能只靠本索引文档本身算出来，必须去**别的索引**查关联数据，再填回来。”*

也就是说，连**数据迁移**时都要做“跨索引关联查询”。

  - 写yaml配置文件

```yaml
'attributes.inventorySourceId':
    source: initialAttributes
    handler:
        type: FOREIGN
        properties:
            index: index_data_source
            path: attributes.vm.inventorySourceId
            jsonReference: dataSourceId
```

4.2.2. 举例二：事件监听机制

- "父对象更新" → "MQ 消息投递" → "所有引用文档更新完成"
- **执行流程**：
  a. 监听 RabbitMQ 上 `INFRASTRUCTURE_OBJECT ... OPERATION.UPDATED` 消息
  b. 反序列化出 `before` 和 `after` 两个对象
  c. 比对 `before` vs `after`，判断哪些字段发生了变化
  d. 查 ReferenceMappings 配置，找出哪些其他 ES 索引的文档里嵌套引用了这个对象
    i. ReferenceMappings – 静态"外键约束表"
  e. **分页遍历**所有相关文档（`PAGE_SIZE = 100`），逐一更新其中的嵌套引用字段
- 弱点：一致性窗口（时间差）

4.3. 可配置的 Refresh 策略 — 解决"NRT 延迟"

- 两层模式设计：
  - 【策略层】EsRefreshStrategyFactory：某个**索引**整体上应该采用哪种 refresh 模式

```java
public enum RefreshStrategy {
    REFRESH_BEFORE_SEARCH,    // 默认：搜索前强制刷新索引
    REFRESH_AFTER_WRITE,
    DELEGATE_TO_ELASTICSEARCH;
}
```

  - 【执行层】Query.getRefresh()： 从策略层的配置转换到实际执行判断 – 这一次 query 要不要在查询前显式 refresh在关键操作前会主动调用 refresh
    - query.setRefresh(true/false)

- 局限：频繁 refresh 会降低写入性能

4.4. Postgres Key-Value Store — 补充 ES 在事务性操作上的不足

- 对于需要强一致性和事务保证的 KV 操作，项目引入了 PostgreSQL 实现的 PgDistributedMap.java，以及 TTL 过期机制。这是对 ES 的一个重要补充。

4.5. 事件监听机制 — 弥补无触发器/外键

5. 仍未解决的弊端

1. **无多文档事务**：OCC 只能保证单文档级别的并发安全。如果一个业务操作需要原子性地更新多个文档（跨 index），仍然无法保证。反范式化更新可能导致部分成功、部分失败的中间状态。
2. **反范式化的数据一致性窗口**：Denormalizer 是异步/后置触发的，在源文档更新后、反范式化完成前存在 **短暂的数据不一致窗口**。如果 Denormalizer 执行失败，冗余数据将不一致且没有自动修复机制。
3. **Refresh 的性能代价**：默认 `REFRESH_BEFORE_SEARCH` 意味着每次搜索前都会强制 refresh，在高并发读场景下会严重影响 ES 写入性能（refresh 是昂贵操作）。这本质上是用性能换一致性。
4. **Schema 变更仍然受限**：SchemaPatch 只能添加字段或 reindex，**无法修改已有字段的类型**。例如把一个 `keyword` 改成 `text` 必须创建新索引并 reindex 全量数据，代价很大。
5. **无引用完整性保证**：删除一个被其他文档引用的实体时，没有外键约束来阻止，orphan references 完全靠应用层代码来防范。
6. **深度分页性能差**：ES 的 `from + size` 分页在偏移量大时性能极差。虽然代码中实现了 `SerialAccessPageRequest`（基于 `search_after`）来缓解，但随机跳页仍然是性能瓶颈。
7. **无悲观锁/SELECT FOR UPDATE**：对于需要排他锁的场景（如唯一资源分配），只能依靠 Postgres KV Store 的 PgDistributedMap.java 或外部协调机制，ES 本身无法提供。
8. **存储开销大**：ES 为了搜索性能会构建倒排索引、存储 `_source`、doc values 等，同等数据量下存储开销远大于关系型数据库。

6. Lucene

6.1. `searcher / segment` 是什么

这是 Elasticsearch / Lucene 的底层概念。你可以先把它简单理解成：

- **segment**：一批已经“整理好、可搜索”的倒排索引文件
- **searcher**：当前用来查询的“搜索视图”，它会看到一组 segment
- for **search()** 场景
- 好处：查询更快、并发更简单、减少锁竞争、提高稳定性

6.1.1. segment

- 做法：Lucene 不会把所有索引数据放在一个大文件里反复修改。而是：
  a. 新写入一批文档
  b. 生成一个新的小索引文件块
  c. 这个文件块就叫 **segment**
- 特点：
  - segment 基本不可变
  - 新文档不会直接改旧 segment，删除、更新通常也是“标记删除 + 新增版本”
    - 追加新数据 + 标记旧数据失效，以更新为例

2. 更新文档

比如原来旧文档在旧 `segment A` 里：

```json
{id: "123", name: "vm-01"}
```

现在把它改成：

```json
{id: "123", name: "vm-02"}
```

Lucene/ES 通常不会直接到 `segment A` 里把 `vm-01` 改成 `vm-02`。而是：

- 先把旧版本标记为 deleted
- 再把新版本写入新的 segment

所以结果变成：

- `segment A` 里旧版本还在，但被标记删除
- `segment B` 里有新版本

  - 后台再做 **merge**，把多个小 segment 合并成更大的 segment –> 是一次重写和整理索引
    - 提高查询效率

    - 步骤：
        1. 读取多个旧 segment 的内容
        2. 把还有效的文档重新写入一个新的更大 segment
        3. 把已删除/过期/旧版本的数据清理掉
        4. 新的大 segment 建好后，再把旧的小 segment 删除

6.1.2. searcher

- 工作原理：
    a. search 请求来的时候
    b. Elasticsearch/Lucene 用一个 searcher 去查
    c. 这个 searcher 只会看到它打开时那一刻可见的 segment
- 所以如果刚写入的数据还没变成新的可见 segment， 当前 searcher 就看不到。

6.1.3. 整体流程

流程可以串起来理解成：

1. 写入文档
2. refresh 后变成新的可搜索小 segment
3. searcher 能看到它
4. 随着小 segment 越来越多
5. 后台再 merge 成更大的 segment

所以：

- **refresh**：让新数据“能被搜到”
- **merge**：让索引“更高效、更整洁”